

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №1.

Тема: «Классы и объекты. Инкапсуляция»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

1. Реализовать определение нового класса. Для демонстрации работы с объектами написать главную функцию. Продемонстрировать разные способы создания объектов и массивов объектов.
2. Структура-пара – структура с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такой структуры. Во всех заданиях должны присутствовать:
 - a. метод инициализации `Init` (метод должен контролировать значения аргументов на корректность);
 - b. ввод с клавиатуры `Read`;
 - c. вывод на экран `Show`.
3. Реализовать внешнюю функцию `make_тип()`, где `тип` – тип реализуемой структуры. Функция должна получать значения для полей структуры как параметры функции и возвращать структуру как результат. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

Поле `first` – положительное целое число, целая часть числа, поле `second` – положительное дробное число, дробная часть числа. Реализовать метод `multiply(double k)` – умножение на вещественное число `k`.

Анализ задачи:

Описываем класс `Pair`

- Два приватных поля: `int first (>0)`, `double second ([0;1))`
- Конструктор по умолчанию (обнуляет поля)

Реализуем обязательные методы:

- `Init(f, s)` — инициализация с проверкой: если данные неверные → возвращает `false`
- `Read()` — ввод с клавиатуры → вызывает `Init` для валидации
- `Show()` — вывод на экран в формате `first + second`
- `multiply(k)` — умножает полное число на `k`, пересчитывает `first` и `second`

Пишем внешнюю функцию `make_Pair(f, s)`

- Создаёт объект, вызывает `Init`
- Если ошибка → выводит сообщение и `exit(1)` (завершает программу)

В main демонстрируем:

- 3 способа создания объекта: через Init, через make_Pair, через Read
- Работу multiply (умножение и пересчёт частей)
- Статический массив: Pair arr[3] (память в стеке)
- Динамический массив: new Pair[2] + delete[] (память в куче)

Код:

```
#include <iostream>
#include <cstdlib>
using namespace std;

// КЛАСС Pair
class Pair {
private:
    int first;      // целая часть (> 0)
    double second;  // дробная часть (0 <= second < 1)

public:
    // Конструктор по умолчанию
    Pair() : first(0), second(0.0) {}

    // а. Метод инициализации с проверкой
    bool Init(int f, double s) {
        if (f > 0 && s >= 0.0 && s < 1.0) {
            first = f;
            second = s;
            return true;
        }
        return false;
    }

    // б. Ввод с клавиатуры
    void Read() {
        int f;
        double s;
        cout << "Целая часть (>0): ";
        cin >> f;
```

```

    cout << "Дробная часть (0...1): ";
    cin >> s;

    if (!Init(f, s)) {
        cout << "Ошибка: некорректные значения!\n";
    }
}

// с. Вывод на экран
void Show() const {
    cout << first << " + " << second << endl;
}

// Умножение на число k
void multiply(double k) {
    double total = first + second;
    total *= k;
    first = static_cast<int>(total);
    second = total - first;
}
};

// ВНЕШНЯЯ ФУНКЦИЯ make_Pair
Pair make_Pair(int f, double s) {
    Pair p;
    if (!p.Init(f, s)) {
        cerr << "Ошибка make_Pair: некорректные параметры!\n";
        exit(1);
    }
    return p;
}

int main() {
    setlocale(LC_ALL, "ru");

    // Способ 1: конструктор + Init
    cout << "Способ 1: Init\n";
    Pair p1;

```

```
p1.Init(5, 0.25);
p1.Show();

// Способ 2: фабричная функция
cout << "\nСпособ 2: make_Pair\n";
Pair p2 = make_Pair(10, 0.75);
p2.Show();

// Способ 3: ввод с клавиатуры
cout << "\nСпособ 3: Read\n";
Pair p3;
p3.Read();
cout << "Вы ввели: ";
p3.Show();

// Умножение
cout << "\nУмножение p1 на 2.5\n";
p1.multiply(2.5);
p1.Show();

// Статический массив
cout << "\nСтатический массив\n";
Pair arr[3];
arr[0].Init(1, 0.1);
arr[1].Init(2, 0.2);
arr[2].Init(3, 0.3);
for (int i = 0; i < 3; i++) {
    cout << "arr[" << i << "] = ";
    arr[i].Show();
}

// Динамический массив
cout << "\nДинамический массив\n";
Pair* dynArr = new Pair[2];
dynArr[0] = make_Pair(100, 0.99);
dynArr[1] = make_Pair(200, 0.01);
for (int i = 0; i < 2; i++) {
    cout << "dynArr[" << i << "] = ";
```

```

        dynArr[i].Show();
    }
    delete[] dynArr;

    return 0;
}

```

Результат выполнения:

```

Способ 1: Init
5 + 0.25

Способ 2: make_Pair
10 + 0.75

Способ 3: Read
Целая часть (>0): 56
Дробная часть (0...1): 0.8
Вы ввели: 56 + 0.8

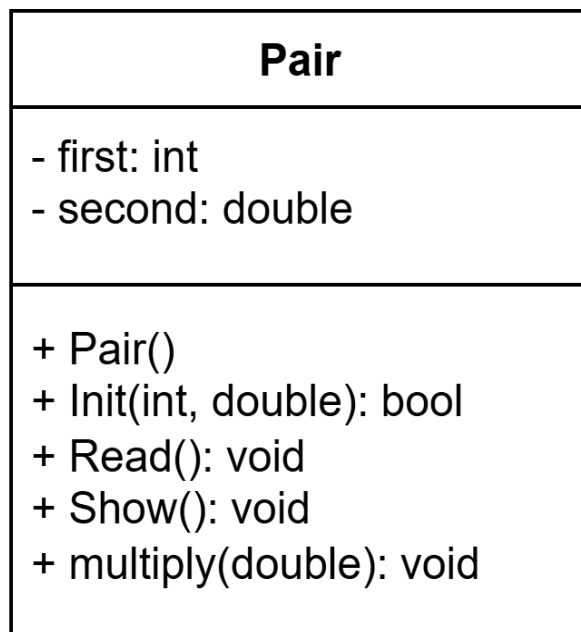
Умножение p1 на 2.5
13 + 0.125

Статический массив
arr[0] = 1 + 0.1
arr[1] = 2 + 0.2
arr[2] = 3 + 0.3

Динамический массив
dynArr[0] = 100 + 0.99
dynArr[1] = 200 + 0.01

```

UML:



Вопросы:

1. Класс — это определённый пользователем абстрактный тип данных. Он моделирует реальный объект, объединяя данные и функции для работы с ними.
2. Объект — это переменная типа класс. Класс объявляется один раз, а объектов на его основе можно создать множество.
3. Поля класса также называют атрибутами. Это данные, которые хранятся в объекте.
4. Функции внутри класса называются методами.
5. Спецификаторы `private` и `public` управляют тем, из каких частей программы видны те или иные элементы класса.
6. Спецификатор `public` описывает интерфейс класса. Элементы, объявленные после `public`, доступны всем внешним функциям и объектам.
7. Спецификатор `private` скрывает элементы внутри класса. К ним имеют доступ только методы самого класса, но не внешний код.
8. Если описание класса начинается со слова `class`, все поля и методы по умолчанию будут скрытыми (`private`).
9. Если описание класса начинается со слова `struct`, все поля и методы по умолчанию будут общедоступными (`public`).
10. Интерфейс класса должен быть объявлен как `public`, чтобы обеспечить доступ к методам класса извне (например, из функции `main`).
11. Если поле открыто (`public`), можно менять его напрямую. Если поле скрыто (`private`), необходимо использовать специальные методы-модификаторы.
12. Для скрытых полей значения получают через методы-селекторы. Для открытых полей допустим прямой доступ.
13. Класс описан как `struct Student { string name; int group; ... }`; объект `Student *s = new Student`; обращение к полю `name`: `s->name` или `(*s).name`, потому что `struct` даёт публичный доступ, а `s` — указатель.
14. Класс описан как `struct Student { string name; int group; ... }`; объект `Student s`; обращение к полю `name`: `s.name` (через точку).
15. Класс описан как `class Student { string name; int group; ... }`; объект `Student *s = new Student`; поле `name` по умолчанию `private`, поэтому прямой доступ запрещён. Нужно использовать публичные методы, например `s->GetName()`.
16. Класс описан как `class Student { string name; int group; public: ... }`; объект `Student s`; поле `name` находится в закрытой части, прямое обращение `s.name` невозможно — только через публичные методы.

17. Класс описан как `class Student { public: char* name; int group; };` объект `Student *s = new Student;` поле `name` открыто, `s` — указатель, доступ через `s->name` или `(*s).name`.